

VAN LANG UNIVERSITY

Faculty of Information Technology / FIT Laboratory

Q-VisionFrame

A Simulation-Based Hybrid Quantum–Classical Framework for Corruption-Robust Object Detection

Khung lai lượng tử–cổ điển dựa trên mô phỏng
cho nhận diện đối tượng bền vững trước nhiễu ảnh

Research Proposal | Architecture
and Mathematical Formulation

Five-Dataset Benchmark, Robustness, Sobol
Sensitivity, and Non-parametric Statistics

Researcher: Nguyễn Thành Phong

Laboratory: FITLab 01 and FITLab 02

Research type: Simulation-based controlled empirical study

Target domain: Hybrid quantum machine learning and robust object
detection

Date: August 2026

This proposal does not claim quantum advantage, quantum speedup, or hardware-level superiority. It tests whether a small parameterized quantum feature gate provides a reproducible robustness–accuracy trade-off beyond parameter-matched classical controls.

Tóm tắt nội dung

Đề xuất này trình bày Q-VISIONFRAME, một khung thực nghiệm lai lượng tử–cổ điển nhằm kiểm định có kiểm soát tác động của mạch lượng tử tham số lên độ bền vững của mô hình nhận diện đối tượng trước nhiễu ảnh và dịch chuyển miền. Thành phần trung tâm là *Parameterized Quantum Feature Gate* (PQFG), được đặt tại mức đặc trưng ngữ nghĩa cao của neck hoặc bottleneck. Feature tensor không được ánh xạ trực tiếp vào trạng thái lượng tử; thay vào đó, tensor được tổng hợp không gian, nén có học, chuẩn hóa thành góc quay, xử lý bởi mạch lượng tử nhỏ từ 4 đến 8 qubit, rồi giải mã thành vector điều chế kênh theo cơ chế residual gating.

Nghiên cứu sử dụng năm tập dữ liệu: PASCAL VOC, MS COCO, ExDark, BDD100K và Cityscapes. Các phiên bản corruption dẫn xuất (PASCAL-C, COCO-C và Cityscapes-C) được sử dụng như bộ benchmark robustness, không được tính là tập dữ liệu độc lập. Thực nghiệm bao gồm đối chứng classical cùng ngân sách tham số, ablation theo số qubit, độ sâu mạch, kiểu mã hóa, topology entanglement, số shots và vị trí chèn module. Chỉ số đánh giá gồm COCO-style AP, mean performance under corruption, relative performance under corruption, relative corruption degradation, hiệu chuẩn độ tin cậy, chi phí tính toán và độ ổn định gradient.

Để tránh kết luận dựa trên một cấu hình đơn lẻ, proposal kết hợp phân tích độ nhạy toàn cục Sobol với thiết kế thực nghiệm nhiều seed. So sánh cặp sử dụng Wilcoxon signed-rank khi giả định chuẩn không bảo đảm; so sánh nhiều mô hình sử dụng Friedman test và hậu kiểm Wilcoxon có hiệu chỉnh Holm. Kết quả được báo cáo bằng hiệu số, khoảng tin cậy bootstrap và effect size, thay vì chỉ dựa vào p-value. Mục tiêu của nghiên cứu là đưa ra bằng chứng trung lập và tái lập được về giá trị hoặc giới hạn của feature gating dựa trên PQC trong dense prediction khi chỉ sử dụng simulator.

Mục lục

1	Bối cảnh và động lực nghiên cứu	1
1.1	Vấn đề của hướng đề tài ban đầu	1
2	Tên đề tài, mục tiêu và phạm vi	1
2.1	Tên đề tài đề xuất	1
2.2	Mục tiêu tổng quát	2
2.3	Mục tiêu cụ thể	2

2.4	Ngoài phạm vi	2
3	Câu hỏi và giả thuyết nghiên cứu	2
4	Kiến trúc chính của Q-VisionFrame	3
4.1	Tổng quan hệ thống	3
4.2	Tensor-to-qubit interface	4
4.3	Mã hóa và mạch lượng tử tham số	4
4.4	Measurement và feature modulation	5
4.5	Hàm mất mát	5
5	Năm tập dữ liệu và vai trò trong benchmark	6
6	Tiền xử lý, tăng cường và tạo dữ liệu nhiễu	7
6.1	Chuẩn hóa dữ liệu	7
6.2	Tăng cường dữ liệu huấn luyện	7
6.3	Bộ corruption	8
6.4	Robust training regimes	8
7	Bộ benchmark và hệ thống đối chứng	9
7.1	Baseline detector	9
7.2	Đối chứng bắt buộc	9
7.3	Ma trận benchmark	10
8	Thang đánh giá	10
8.1	COCO-style detection metrics	10
8.2	Robustness metrics	11
8.3	Calibration và reliability	11
8.4	Efficiency và trainability	11
9	Phân tích độ nhạy toàn cục Sobol	12
9.1	Mục tiêu	12
9.2	Không gian tham số Sobol	13
10	Thiết kế thống kê: Wilcoxon, Friedman và bootstrap	13
10.1	Đơn vị phân tích	14
10.2	Wilcoxon signed-rank	14
10.3	Friedman test	14
10.4	Hiệu chỉnh đa so sánh	14

10.5 Bootstrap confidence intervals	15
10.6 Ngưỡng ý nghĩa thực tiễn	15
11 Ablation study	15
12 Tài nguyên và triển khai trên FITLab 01/02	16
12.1 Khảo sát phần cứng bắt buộc	16
12.2 Stack phần mềm	17
12.3 Phân công tài nguyên	17
12.4 Ước lượng tài nguyên	17
13 Quy trình huấn luyện	18
13.1 Ba giai đoạn	18
13.2 Reproducibility controls	18
14 Kế hoạch phân tích kết quả	18
15 Tiêu chí thành công và quy tắc kết luận	19
16 Rủi ro và phương án giảm thiểu	20
17 Lộ trình thực hiện	21
18 Đóng góp dự kiến	21
19 Kết luận	22

Danh sách bảng

1	Năm tập dữ liệu chính và vai trò trong nghiên cứu.	6
2	Các mô hình đối chứng nhằm xác định nguồn gốc của cải thiện.	9
3	Ma trận benchmark tối thiểu.	10
4	Các biến đầu vào cho Sobol sensitivity analysis.	13
5	Các ablation chính.	15
6	Phân bố thực nghiệm đề xuất.	17
7	Rủi ro nghiên cứu và biện pháp xử lý.	20
8	Lộ trình 24 tuần đề xuất.	21

Danh sách hình vẽ

- 1 Kiến trúc mức cao. PQC không xử lý từng pixel; nó xử lý descriptor kích thước thấp và sinh vector điều chế đặc trưng. 3

1 Bối cảnh và động lực nghiên cứu

Các detector hiện đại đạt độ chính xác cao trên dữ liệu sạch nhưng thường suy giảm mạnh trước noise, blur, fog, thay đổi độ sáng, tương phản và nén ảnh. Benchmark corruption cho object detection đã chỉ ra rằng hiệu năng của nhiều detector có thể chỉ còn khoảng 30–60% so với dữ liệu sạch trong một số điều kiện [4]. Vì vậy, robustness không thể được suy ra trực tiếp từ clean mAP.

Song song đó, PennyLane hỗ trợ biến một QNode thành lớp PyTorch thông qua TorchLayer, cho phép gradient lan truyền trong mô hình lai [1, 2]. Điều này làm cho nghiên cứu kiến trúc hybrid khả thi ở mức phần mềm. Tuy nhiên, simulator state-vector có độ phức tạp bộ nhớ tăng theo số trạng thái 2^q ; do đó, đề tài phải giới hạn số qubit, không xử lý từng pixel hoặc anchor bằng một circuit riêng, và không tuyên bố tốc độ lượng tử hay lợi thế phần cứng.

1.1 Vấn đề của hướng đề tài ban đầu

Hướng ban đầu có ba rủi ro khoa học chính:

1. Đồng nhất “không gian Hilbert lớn” với chất lượng biểu diễn tốt hơn, trong khi đây chưa phải bằng chứng về quantum advantage.
2. Gọi module là quantum attention dù không xây dựng toán tử query–key–value hoặc một định nghĩa attention tương ứng.
3. Đề xuất ánh xạ feature tensor rất lớn trực tiếp vào số qubit nhỏ mà không mô tả rõ tổn thất thông tin và chi phí state preparation.

Proposal sửa các điểm trên bằng cách chuyển trọng tâm sang **controlled simulation-based study**. PQC được xem là một phép biến đổi phi tuyến có inductive bias khác biệt, cần được kiểm chứng công bằng với các phép biến đổi classical có cùng input, output, số tham số và vị trí tích hợp.

2 Tên đề tài, mục tiêu và phạm vi

2.1 Tên đề tài đề xuất

Q-VisionFrame: A Simulation-Based Hybrid Quantum–Classical Framework for Corruption-Robust Object Detection.

2.2 Mục tiêu tổng quát

Xây dựng và đánh giá một framework lai lượng tử–cổ điển tái lập được nhằm xác định liệu một PQC nhỏ dùng cho feature gating có cải thiện độ bền vững trước common corruptions và natural domain shifts so với detector nền và các module classical tương đương hay không.

2.3 Mục tiêu cụ thể

1. Thiết kế PQFG có 4–8 qubit, mạch nông và residual stabilization.
2. Xây dựng giao diện tensor-to-qubit khả thi về tính toán.
3. Benchmark trên năm dataset, bao phủ dữ liệu sạch, low-light, giao thông và urban scenes.
4. Tạo benchmark corruption theo loại nhiễu và severity.
5. So sánh với Identity, MLP gate, SE, ECA, frozen random PQC, PQC không entanglement và classical trigonometric gate.
6. Đánh giá trainability, resource cost, gradient stability và finite-shot sensitivity.
7. Phân tích độ nhạy hyperparameter bằng Sobol indices.
8. Thực hiện kiểm định Wilcoxon, Friedman và hậu kiểm có hiệu chỉnh đa so sánh.

2.4 Ngoài phạm vi

Nghiên cứu không tuyên bố quantum speedup, practical quantum advantage, real-time inference trên QPU, khả năng mở rộng lên hàng trăm qubit hoặc robustness đối với adversarial attacks nếu không có thí nghiệm adversarial riêng.

3 Câu hỏi và giả thuyết nghiên cứu

RQ1.

PQFG có cải thiện corrupted mAP và relative robustness so với detector nền hay không?

RQ2.

Lợi ích, nếu có, còn tồn tại khi so với classical gate được match số tham số và cùng giao diện tensor-to-gate hay không?

RQ3.

Số qubit, circuit depth, encoding, entanglement, shots và vị trí chèn ảnh hưởng như thế nào đến accuracy, robustness và trainability?

RQ4.

PQC có cung cấp giá trị ngoài bottleneck, residual gating và regularization thông thường hay không?

RQ5.

Kết quả trên analytic simulator có ổn định khi dùng finite shots và simulated device noise hay không?

Giả thuyết được đặt trung lập:

$$H_{0,1} : \text{median}(\Delta \text{mPC}) = 0, \quad (1)$$

$$H_{A,1} : \text{median}(\Delta \text{mPC}) \neq 0, \quad (2)$$

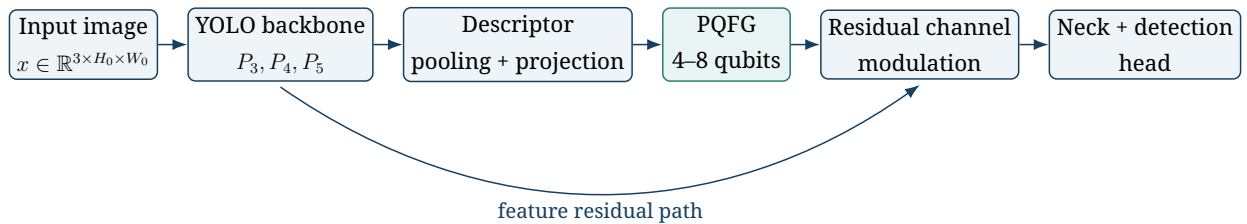
$$H_{0,2} : \text{median}(\Delta \text{rPC}) = 0, \quad (3)$$

$$H_{A,2} : \text{median}(\Delta \text{rPC}) \neq 0, \quad (4)$$

trong đó Δ được tính giữa PQFG và đối chứng tương ứng trên cùng seed, dataset và corruption condition.

4 Kiến trúc chính của Q-VisionFrame

4.1 Tổng quan hệ thống



Hình 1: Kiến trúc mức cao. PQC không xử lý từng pixel; nó xử lý descriptor kích thước thấp và sinh vector điều chế đặc trưng.

4.2 Tensor-to-qubit interface

Cho feature map tại mức P_3 :

$$\mathbf{X} \in \mathbb{R}^{B \times C \times H \times W}. \quad (5)$$

Descriptor kênh kết hợp global average pooling và global max pooling:

$$\mathbf{z}_{\text{avg}} = \frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W \mathbf{X}_{::,h,w}, \quad (6)$$

$$\mathbf{z}_{\text{max}} = \max_{h,w} \mathbf{X}_{::,h,w}, \quad (7)$$

$$\mathbf{z} = [\mathbf{z}_{\text{avg}}; \mathbf{z}_{\text{max}}] \in \mathbb{R}^{B \times 2C}. \quad (8)$$

Một projector cổ điển nén descriptor xuống q chiều:

$$\mathbf{u} = \mathbf{W}_2 \phi(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2, \quad \mathbf{u} \in \mathbb{R}^{B \times q}, \quad (9)$$

trong đó $q \in \{4, 6, 8\}$ và ϕ là SiLU hoặc GELU. Góc mã hóa được chặn:

$$\tilde{\mathbf{u}} = \pi \tanh(\mathbf{u}). \quad (10)$$

Cơ chế này tránh đưa trực tiếp hàng triệu phần tử feature map vào simulator.

4.3 Mã hóa và mạch lượng tử tham số

Trạng thái ban đầu là $|0\rangle^{\otimes q}$. Với angle encoding:

$$|\psi(\tilde{\mathbf{u}})\rangle = \prod_{i=1}^q R_y(\tilde{u}_i) R_z(\gamma_i \tilde{u}_i) |0\rangle^{\otimes q}, \quad (11)$$

trong đó γ_i có thể cố định bằng 1 hoặc được học.

Mỗi variational layer ℓ gồm local rotations và entanglement ring:

$$U_\ell(\boldsymbol{\theta}_\ell) = U_{\text{ring}} \prod_{i=1}^q R_z(\theta_{\ell i}^z) R_y(\theta_{\ell i}^y), \quad (12)$$

$$U_{\text{ring}} = \prod_{i=1}^{q-1} \text{CNOT}_{i,i+1} \cdot \text{CNOT}_{q,1}. \quad (13)$$

Toàn mạch có độ sâu L :

$$U(\boldsymbol{\theta}, \tilde{\mathbf{u}}) = \prod_{\ell=1}^L U_{\ell}(\boldsymbol{\theta}_{\ell}) U_{\text{enc}}(\tilde{\mathbf{u}}). \quad (14)$$

Data re-uploading được khảo sát bằng cách lặp lại U_{enc} giữa các variational layers.

4.4 Measurement và feature modulation

Đầu ra lượng tử là các expectation values:

$$m_i = \langle \psi | U^{\dagger} Z_i U | \psi \rangle, \quad (15)$$

$$m_{i,i+1} = \langle \psi | U^{\dagger} (Z_i Z_{i+1}) U | \psi \rangle. \quad (16)$$

Vector measurement $\mathbf{m}$ được giải mã về C kênh:

$$\mathbf{g} = \sigma(\mathbf{W}_g \mathbf{m} + \mathbf{b}_g) \in (0, 1)^{B \times C}. \quad (17)$$

Feature output:

$$\mathbf{Y} = \mathbf{X} \odot (1 + \alpha \mathbf{g}), \quad (18)$$

trong đó α là scalar trainable, khởi tạo bằng 0 hoặc 10^{-3} . Khởi tạo gần identity giúp không phá hỏng pretrained features trong các epoch đầu.

4.5 Hàm mất mát

Loss detector tổng quát:

$$\mathcal{L}_{\text{det}} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}} \mathcal{L}_{\text{dfl}}. \quad (19)$$

Có thể thêm consistency loss giữa ảnh sạch x và ảnh corrupted $\tilde{x}$:

$$\mathcal{L}_{\text{cons}} = D_{\text{KL}}(p(y|x) \| p(y|\tilde{x})) + \lambda_f \|h(x) - h(\tilde{x})\|_2^2. \quad (20)$$

Loss cuối:

$$\mathcal{L} = \mathcal{L}_{\text{det}} + \beta \mathcal{L}_{\text{cons}}. \quad (21)$$

Consistency loss phải được áp dụng giống nhau cho baseline classical và hybrid để giữ fairness.

5 Năm tập dữ liệu và vai trò trong benchmark

Bảng 1: Năm tập dữ liệu chính và vai trò trong nghiên cứu.

Dataset	Quy mô tiêu biểu	Nhãn	Vai trò	Protocol đề xuất
PASCAL VOC 2007+2012	Khoảng 20 lớp đối tượng; tập train/val/test chuẩn	Bounding boxes, classes	Development benchmark, ablation nhanh, nhiều seed	Train VOC07+12 trainval; test VOC07 test; tạo PASCAL-C cho corruption
MS COCO 2017	Quy mô lớn, scenes phức tạp, 80 lớp detection phổ biến	Bounding boxes, masks, categories	Main clean benchmark và COCO-style AP	Train2017, val2017; subset cho sweep; full validation cho final
ExDark	7,363 ảnh low-light, 12 lớp, 10 điều kiện ánh sáng [7, 8]	Image-level và bounding boxes	Natural low-light shift; không cần tạo darkness nhân tạo để thay thế hoàn toàn	Fine-tune hoặc cross-domain evaluation theo class mapping được công bố
BDD100K	100K driving videos, đa dạng thời tiết, thời gian và địa lý [9]	Detection và nhiều task khác	Natural weather/time shift trong giao thông	Chọn detection frames; stratify theo weather/time; evaluate day/night/rainy subsets

Dataset	Quy mô tiêu biểu	Nhãn	Vai trò	Protocol đề xuất
Cityscapes	5,000 ảnh fine annotation và 20,000 ảnh coarse từ 50 thành phố [10]	Pixel/instance labels; có thể chuyển instance masks thành boxes	Urban-domain transfer và Cityscapes-C robustness	Dùng fine train/val; derive boxes từ instances; benchmark Cityscapes-C

COCO được thiết kế cho object recognition trong bối cảnh tự nhiên và có hàng triệu instance annotations [5]. ExDark cung cấp ảnh thực dưới nhiều kiểu low-light [7]. BDD100K có đa dạng môi trường và thời tiết cho driving scenes [9]. Cityscapes cung cấp urban scenes với annotation instance chất lượng cao [10]. Các bộ corruption PASCAL-C, COCO-C và Cityscapes-C là biến thể đánh giá dẫn xuất từ benchmark robustness [4].

6 Tiền xử lý, tăng cường và tạo dữ liệu nhiễu

6.1 Chuẩn hóa dữ liệu

- Chuyển annotation về định dạng COCO JSON thống nhất; lưu mapping class giữa dataset.
- Không gộp score giữa các dataset có taxonomy khác nhau nếu chưa định nghĩa common-class subset.
- Kiểm tra box hợp lệ, ảnh hỏng, duplicate hashes và leakage giữa train/validation/test.
- Resize theo detector baseline, ví dụ short-side hoặc letterbox tới 640×640 ; giữ aspect ratio.
- Lưu checksum của raw data, converted annotations và corruption manifests.

6.2 Tăng cường dữ liệu huấn luyện

Tập augmentation được chia thành ba nhóm:

1. **Geometric:** horizontal flip, random scale, translation, crop có bảo toàn box, perspective nhẹ.
2. **Photometric:** brightness, contrast, gamma, hue/saturation, color temperature.
3. **Compositional:** Mosaic, MixUp hoặc Copy-Paste chỉ khi baseline hỗ trợ và phải dùng cùng xác suất cho mọi mô hình.

Để tránh benchmark leakage, corruption dùng cho validation/test được tạo offline hoặc deterministic theo image ID, corruption type, severity và seed. Augmentation huấn luyện không được dùng chính xác cùng realization với corruption test.

6.3 Bộ corruption

Theo tinh thần ImageNet-C và robust detection benchmark [3, 4], nghiên cứu dùng 15 loại corruption, chia nhóm:

- **Noise:** Gaussian, shot, impulse.
- **Blur:** defocus, glass, motion, zoom.
- **Weather:** snow, frost, fog.
- **Digital/photometric:** brightness, contrast, elastic transform, pixelate, JPEG compression.

Mỗi loại có severity $s \in \{1, 2, 3, 4, 5\}$. Annotation box không thay đổi đối với corruption photometric; với geometric corruption, phải biến đổi box đồng bộ hoặc loại khỏi benchmark nếu không thể xác định chính xác.

6.4 Robust training regimes

Ba regime được so sánh:

1. Clean-only training.
2. Mixed clean-corruption training với xác suất p_c .
3. Curriculum corruption, severity tăng theo epoch.

Mọi regime phải áp dụng cho cả baseline và hybrid. Điều này tách tác động của robustness augmentation khỏi tác động của PQC.

7 Bộ benchmark và hệ thống đối chứng

7.1 Baseline detector

Giai đoạn phát triển dùng một detector nhỏ và dễ chỉnh sửa, ví dụ YOLO-family nano/small hoặc RT-DETR-R18. Một commit cụ thể phải được cố định. Full study không nên trộn nhiều codebase trong cùng sweep.

7.2 Đối chứng bắt buộc

Bảng 2: Các mô hình đối chứng nhằm xác định nguồn gốc của cải thiện.

ID	Mô hình
B0	Detector baseline, không thêm gate
B1	Detector + Squeeze-and-Excitation
B2	Detector + ECA hoặc CBAM
B3	Detector + MLP gate cùng input/output với PQFG
B4	Parameter-matched MLP gate
B5	Classical trigonometric gate $[\sin(Wx), \cos(Wx)]$
B6	Frozen random PQC, chỉ train decoder gate
B7	Trainable PQC không entanglement
B8	Trainable PQFG với entanglement

Parameter matching không đồng nghĩa compute matching. Do đó phải báo cáo đồng thời số tham số, wall-clock time, peak VRAM/RAM, circuit evaluations, latency và throughput.

7.3 Ma trận benchmark

Bảng 3: Ma trận benchmark tối thiểu.

Benchmark Dataset		Mục tiêu	Chỉ số chính
Clean in-domain	VOC, COCO	Accuracy chuẩn	AP, AP50, AP75
Synthetic corruption	PASCAL-C, COCO-C, Cityscapes-C	Common corruption robustness	mPC, rPC, RCD
Natural low-light	ExDark	Shift ánh sáng thực	AP, class-wise AP, retention
Driving condition shift	BDD100K	Day/night/weather robustness	subgroup AP, worst-group AP
Urban transfer	Cityscapes	Cross-domain generalization	AP, AP50, calibration
Simulator realism	Selected subsets	Finite shots và device noise	metric drop, variance, latency

8 Thang đánh giá

8.1 COCO-style detection metrics

Average Precision tại IoU threshold t :

$$AP_t = \int_0^1 p_t(r) dr. \quad (22)$$

Chỉ số chính:

$$mAP_{0.5:0.95} = \frac{1}{10} \sum_{k=0}^9 AP_{0.50+0.05k}. \quad (23)$$

Báo cáo thêm AP_{50} , AP_{75} , AP_S , AP_M , AP_L .

8.2 Robustness metrics

Với C corruption types và $S = 5$ severity levels:

$$\text{mPC} = \frac{1}{CS} \sum_{c=1}^C \sum_{s=1}^S \text{AP}_{c,s}. \quad (24)$$

Relative performance under corruption:

$$\text{rPC} = \frac{\text{mPC}}{\text{AP}_{\text{clean}}}. \quad (25)$$

Relative corruption degradation:

$$\text{RCD} = \frac{\text{AP}_{\text{clean}} - \text{mPC}}{\text{AP}_{\text{clean}}}. \quad (26)$$

Worst-corruption performance:

$$\text{AP}_{\text{worst}} = \min_{c,s} \text{AP}_{c,s}. \quad (27)$$

Area under corruption-severity curve:

$$\text{AUC}_{\text{sev}} = \frac{1}{C} \sum_{c=1}^C \text{AUC}(s, \text{AP}_{c,s}). \quad (28)$$

8.3 Calibration và reliability

Expected calibration error:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|. \quad (29)$$

Có thể báo cáo negative log-likelihood hoặc Brier score trên matched detections. Calibration phải được đánh giá riêng trên clean và corrupted data.

8.4 Efficiency và trainability

- Trainable parameters và tổng parameters.
- Peak GPU memory, CPU RAM, wall-clock/epoch và total GPU-hours.
- Inference latency, images/s và circuit evaluations/image.

- Quantum gradient norm $\|\nabla_{\theta_q} \mathcal{L}\|_2$.
- Gradient variance theo layer và tỷ lệ gradient gần 0.
- Convergence epoch và failure/NaN rate.

9 Phân tích độ nhạy toàn cục Sobol

9.1 Mục tiêu

Sobol analysis xác định mức đóng góp của từng hyperparameter và tương tác giữa chúng đối với output như mPC, rPC, clean AP và training cost. SALib cung cấp Sobol sampling và estimator cho first-order, total-order và second-order indices [11–13].

Cho mô hình thực nghiệm:

$$Y = f(X_1, \dots, X_D), \quad (30)$$

phân rã phương sai:

$$\text{Var}(Y) = \sum_i V_i + \sum_{i < j} V_{ij} + \dots + V_{1\dots D}. \quad (31)$$

First-order index:

$$S_i = \frac{\text{Var}_{X_i}(\mathbb{E}[Y|X_i])}{\text{Var}(Y)}. \quad (32)$$

Total-order index:

$$S_{T_i} = 1 - \frac{\text{Var}_{X_{\sim i}}(\mathbb{E}[Y|X_{\sim i}])}{\text{Var}(Y)}. \quad (33)$$

Second-order interaction:

$$S_{ij} = \frac{V_{ij}}{\text{Var}(Y)}. \quad (34)$$

9.2 Không gian tham số Sobol

Bảng 4: Các biến đầu vào cho Sobol sensitivity analysis.

Biến	Miền	Mã hóa/ghi chú
q	[4, 8]	Làm tròn về {4, 6, 8}; phân tích bổ sung theo categorical group
L	[1, 4]	Circuit depth nguyên
η_q/η_c	[0.01, 0.5]	Log-uniform khuyến nghị
α_0	[0, 0.1]	Residual scale ban đầu
p_c	[0, 0.7]	Xác suất corruption augmentation
β	[0, 1]	Trọng số consistency loss
shots placement	[128, 2048] P4/P5/P4+P5	Chỉ trong finite-shot phase; analytic là phase riêng Categorical; chạy stratified hoặc surrogate encoding

Full Saltelli design có thể rất tốn kém. Với D biến và không tính second-order, số run xấp xỉ $N(D + 2)$ theo SALib documentation [13]. Vì mỗi run là một detector training, proposal dùng chiến lược hai tầng:

1. Chạy low-fidelity proxy trên VOC subset, ít epoch và 1 seed để fit surrogate model.
2. Thực hiện Sobol trên surrogate; sau đó xác nhận top factors bằng full-fidelity runs và nhiều seed.

Surrogate có thể là Gaussian Process, random forest hoặc gradient boosting. Phải báo cáo cross-validation error của surrogate và không diễn giải Sobol indices nếu surrogate quá kém.

10 Thiết kế thống kê: Wilcoxon, Friedman và bootstrap

10.1 Đơn vị phân tích

Đơn vị lặp độc lập chính là seed. Với mỗi seed, các mô hình được huấn luyện trên cùng split và đánh giá trên cùng image IDs/corruption manifest. Điều này tạo thiết kế paired hoặc repeated-measures.

10.2 Wilcoxon signed-rank

Khi so sánh hai mô hình trên n seed hoặc n matched benchmark blocks, xét hiệu số d_i . Wilcoxon signed-rank kiểm tra median của phân phối hiệu số có bằng 0 hay không. Thống kê dựa trên rank của $|d_i|$ và dấu của d_i :

$$W^+ = \sum_{d_i > 0} R_i, \quad W^- = \sum_{d_i < 0} R_i. \quad (35)$$

Dùng two-sided test theo mặc định; one-sided chỉ dùng nếu được preregister trước khi xem kết quả.

Effect size rank-biserial:

$$r_{rb} = \frac{W^+ - W^-}{W^+ + W^-}. \quad (36)$$

10.3 Friedman test

Với k mô hình đánh giá trên n blocks, Friedman test dùng rank trong từng block. Thống kê:

$$Q = \frac{12n}{k(k+1)} \sum_{j=1}^k \bar{R}_j^2 - 3n(k+1), \quad (37)$$

trong đó $\bar{R}_j$ là tổng hoặc mean rank của mô hình j theo quy ước triển khai. Nếu Friedman có ý nghĩa, thực hiện pairwise Wilcoxon hậu kiểm.

Kendall's W đo effect size/concordance:

$$W_K = \frac{Q}{n(k-1)}. \quad (38)$$

10.4 Hiệu chỉnh đa so sánh

Với m pairwise hypotheses, dùng Holm step-down procedure. Sắp p-value:

$$p_{(1)} \leq \dots \leq p_{(m)}, \quad (39)$$

và so sánh $p_{(i)}$ với $\alpha/(m-i+1)$. Không báo cáo $p = 0$; dùng giá trị scientific notation hoặc $p < \epsilon$ theo độ chính xác phần mềm.

10.5 Bootstrap confidence intervals

Paired bootstrap theo seed hoặc image IDs được dùng để ước lượng CI của ΔAP , ΔmPC và ΔrPC . Hai nguồn bất định phải tách riêng:

- between-seed uncertainty;
- evaluation-set sampling uncertainty.

Không gộp image bootstrap samples như thể chúng là các training runs độc lập.

10.6 Ngưỡng ý nghĩa thực tiễn

Ngoài p-value, proposal đặt minimal practically important difference (MPID), ví dụ:

$$|\Delta mPC| \geq 1.0 \text{ AP point} \quad (40)$$

hoặc ngưỡng được chốt sau pilot nhưng trước final testing. Một kết quả thống kê có ý nghĩa nhưng nhỏ hơn MPID không được gọi là cải tiến thực tiễn mạnh.

11 Ablation study

Bảng 5: Các ablation chính.

Nhóm	Cấu hình	Câu hỏi
Qubit	4, 6, 8	Điểm cân bằng giữa expressivity, chi phí và gradient stability
Circuit depth	1, 2, 4	Mạch sâu hơn có tăng robustness hay gây vanishing gradients?
Encoding	angle, learned angle, data re-uploading; amplitude chỉ thử nhỏ	Encoding nào ổn định và có lợi nhất?

Nhóm	Cấu hình	Câu hỏi
Entanglement	none, linear, ring	Lợi ích có phụ thuộc entanglement hay không?
Measurement	Z_i ; $Z_i + X_i$; local $Z_i Z_{i+1}$	Observable nào cung cấp descriptor hữu ích?
Placement	P4, P5, P4+P5	Vị trí nào mang lại robustness tốt trên chi phí thấp?
Gradient method	backprop/adjoint/parameter shift nếu backend hỗ trợ	Trade-off giữa tính đúng gradient và runtime
Shots	analytic, 128, 512, 1024, 2048	Độ nhạy với sampling noise
Noise model	depolarizing, amplitude damping, readout noise	Kết quả ideal simulator có bền khi mô phỏng nhiễu thiết bị?
Residual scale	0, 10^{-3} , 10^{-2} , 10^{-1}	Identity initialization có giúp ổn định fine-tuning?

12 Tài nguyên và triển khai trên FITLab 01/02

12.1 Khảo sát phần cứng bắt buộc

Trước khi chốt backend, chạy:

```
hostname
nvidia-smi
nvcc --version
python3 --version
lscpu
free -h
df -h
```

Không mặc định `lightning.gpu` hoạt động chỉ vì máy có NVIDIA GPU. Backend phải được kiểm chứng với phiên bản CUDA/cuQuantum/PennyLane tương thích.

12.2 Stack phần mềm

- Ubuntu Linux, NVIDIA driver và CUDA theo server.
- Python 3.10–3.12 tùy compatibility matrix.
- PyTorch, torchvision, detector codebase cố định commit.
- PennyLane và PennyLane-Lightning; `default.qubit` cho correctness tests, `lightning.qubit` cho CPU, `lightning.gpu` nếu tương thích.
- pycocotools, Albumentations/Kornia, OpenCV.
- SciPy, statsmodels, scikit-posthocs, SALib.
- MLflow hoặc Weights & Biases tùy chính sách dữ liệu; luôn lưu local CSV/JSON.
- Docker/Apptainer hoặc Conda lockfile để tái lập.

12.3 Phân công tài nguyên

Bảng 6: Phân bổ thực nghiệm đề xuất.

	FITLab 01	FITLab 02
Vai trò	Classical baselines, data pipeline, clean/corruption training	Quantum sweeps, finite shots, device-noise simulation
Runs	B0–B5, seeds 0–2	B6–B8, seeds 3–4 và quantum ablations
Storage	Dataset mirror, corruption cache, baseline checkpoints	Quantum checkpoints, circuit logs, profiling traces
Parallelism	Experiment-level; nhiều jobs độc lập	Experiment-level; chia theo q , L , shots

12.4 Ước lượng tài nguyên

Vì chưa có thông số chính xác của server, proposal dùng ngân sách theo tầng:

- **Pilot:** VOC subset, 20–30 epochs, 1 seed, $q=4$; mục tiêu correctness và profiling.
- **Development:** VOC full, 3 seeds, 40–100 epochs, $q=4/6/8$.

- **Main:** cấu hình được khóa, 5 seeds, COCO/COCO-C và natural-shift datasets.
- **Statistical confirmation:** chỉ baseline và top 1–2 hybrid configurations được tăng lên 10 seeds nếu tài nguyên cho phép.

Mỗi run ghi GPU-hours, CPU-hours, peak memory và storage. Simulator overhead phải được báo cáo như một kết quả, không ẩn khỏi paper.

13 Quy trình huấn luyện

13.1 Ba giai đoạn

1. **Classical warm-up:** train/fine-tune detector nền và lưu checkpoint.
2. **Quantum gate warm-up:** freeze backbone; train projector, PQC và gate với $\eta_q \approx 0.1\eta_c$.
3. **Joint fine-tuning:** unfreeze neck hoặc các block cuối; learning rate nhỏ, gradient clipping và early stopping theo validation mPC/clean AP trade-off.

13.2 Reproducibility controls

Mỗi run phải lưu:

- seed của Python, NumPy, PyTorch, data sampler và corruption generator;
- Git commit, environment lock, GPU model, driver/CUDA;
- dataset checksum, split manifest, class mapping;
- config YAML, command line, checkpoint hash;
- epoch-wise losses, AP, mPC, gradient norms, time và memory;
- failure log và reason nếu run bị loại.

14 Kế hoạch phân tích kết quả

1. Kiểm tra data integrity và distribution của class/box size.
2. Báo cáo clean AP và efficiency cho mọi model.
3. Báo cáo corruption matrix theo loại và severity.

4. Tính mPC, rPC, RCD, worst-group AP và calibration.
5. Kiểm tra gradient stability và learning curves.
6. Chạy Friedman trên mô hình chính qua matched blocks; nếu có ý nghĩa, chạy Wilcoxon hậu kiểm + Holm.
7. Báo cáo paired differences, 95% CI, rank-biserial effect size và Kendall's W .
8. Chạy Sobol trên proxy/surrogate và xác nhận các yếu tố quan trọng bằng full runs.
9. Phân tích failure cases: missed detections, false positives, small objects, low-light và fog.
10. Diễn giải kết quả âm như một kết luận hợp lệ nếu protocol đủ mạnh.

15 Tiêu chí thành công và quy tắc kết luận

Một cấu hình PQFG chỉ được xem là có giá trị khi thỏa phần lớn các điều kiện:

1. ΔmPC hoặc ΔrPC dương so với baseline và parameter-matched classical gate.
2. 95% CI của paired difference không bao gồm 0 hoặc kết quả Wilcoxon qua được Holm correction.
3. Effect size không ở mức không đáng kể và vượt MPID đã định trước.
4. Clean AP không giảm vượt ngưỡng chấp nhận.
5. Cải thiện xuất hiện trên nhiều corruption groups hoặc ít nhất một natural-shift dataset, không chỉ một corruption đơn lẻ.
6. Gradient ổn định, không có failure rate cao.
7. Chi phí simulator được báo cáo minh bạch.

Nếu PQC không vượt classical control, kết luận phù hợp là: trong phạm vi qubit, circuit, data và compute đã khảo sát, chưa có bằng chứng cho lợi ích robustness riêng của PQC. Không được diễn giải thành quantum advantage.

16 Rủi ro và phương án giảm thiểu

Bảng 7: Rủi ro nghiên cứu và biện pháp xử lý.

Rủi ro	Tác động	Giảm thiểu
Simulator quá chậm	Không đủ seed/ablation	Chỉ dùng descriptor-level PQC; $q \leq 8$; experiment-level parallelism; surrogate Sobol
PQC không hội tụ	Kết quả không ổn định	Identity initialization, shallow circuits, warm-up, gradient diagnostics
Cải thiện do thêm tham số	Attribution sai	Parameter-matched MLP, random PQC, no-entanglement control
Dataset taxonomy khác nhau	So sánh không hợp lệ	Dataset-specific metrics; common-class subset được định nghĩa trước
Corruption leakage	Inflated robustness	Deterministic offline test corruptions, tách augmentation và test manifests
Quá nhiều hypothesis tests	False positives	Friedman gatekeeper, Holm correction, preregister primary endpoints
COCO quá tốn tài nguyên	Không hoàn thành	VOC development, COCO subset sweep, chỉ final configs chạy full
Không có QPU	Reviewer nghi ngờ thực tiễn	Định vị simulation study; finite-shot/noise tests; không tuyên bố hardware advantage

17 Lộ trình thực hiện

Bảng 8: Lộ trình 24 tuần đề xuất.

Tuần	Giai đoạn	Sản phẩm
1–2	Audit server và dữ liệu	Hardware report, environment lock, dataset manifests
3–5	Baseline pipeline	VOC/COCO evaluator, B0–B5 baselines, corruption generator
6–8	Minimal PQFG	Correct forward/backward, $q=4$, gradient tests
9–11	Quantum ablation pilot	q /depth/encoding/placement profiling
12–14	Sobol proxy study	Surrogate, S1/ST/S2 indices, top factors
15–18	Main 5-dataset runs	Locked configurations, 5 seeds, clean/corruption/shift results
19–20	Finite-shot/noise phase	Shot/noise sensitivity and resource profile
21–22	Statistical analysis	Friedman, Wilcoxon-Holm, bootstrap CIs, effect sizes
23–24	Writing/release	Paper draft, code, configs, reproducibility package

18 Đóng góp dự kiến

1. Một module feature gating lượng tử nhỏ, residual-stabilized và khả thi trên simulator cho dense prediction.
2. Một giao diện tensor-to-qubit rõ ràng, không che giấu bước nén dữ liệu.
3. Một benchmark năm dataset kết hợp clean, synthetic corruption và natural shift.
4. Một hệ thống đối chứng cho phép phân biệt tác động PQC với bottleneck/gating/parameter count.

5. Một phân tích trainability và resource cost minh bạch.
6. Một quy trình Sobol + Wilcoxon + Friedman + bootstrap có thể tái sử dụng cho hybrid QML experiments.
7. Một kết luận thực nghiệm trung lập về khả năng hoặc giới hạn của PQC-based feature modulation.

19 Kết luận

Q-VISIONFRAME được định vị là một nghiên cứu dựa trên mô phỏng, không phải bằng chứng về lợi thế phần cứng lượng tử. Giá trị khoa học nằm ở thiết kế module khả thi, benchmark robustness đa miền, đối chứng công bằng, phân tích độ nhạy toàn cục và thống kê lặp lại. Việc chỉ dùng simulator vẫn có uy tín khi các tuyên bố được giới hạn đúng, chi phí được báo cáo đầy đủ và kết quả được so sánh với classical controls mạnh. Proposal này chuyển ý tưởng ban đầu từ một tuyên bố “quantum improves YOLO” sang một câu hỏi nghiên cứu có thể bác bỏ, kiểm chứng và tái lập.

Tài liệu

- [1] PennyLane, “qml.qnn.TorchLayer: Converts a QNode to a Torch layer,” official documentation, accessed Aug. 2026. <https://docs.pennylane.ai/en/stable/code/api/pennylane.qnn.TorchLayer.html>
- [2] PennyLane, “PyTorch interface,” official documentation, accessed Aug. 2026. <https://docs.pennylane.ai/en/stable/introduction/interfaces/torch.html>
- [3] D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” in *ICLR*, 2019.
- [4] C. Michaelis, B. Mitzkus, R. Geirhos, E. Rusak, O. Bringmann, A. S. Ecker, M. Bethge, and W. Brendel, “Benchmarking Robustness in Object Detection: Autonomous Driving when Winter is Coming,” arXiv:1907.07484, 2019.
- [5] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in *ECCV*, 2014.
- [6] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, vol. 88, pp. 303–338, 2010.
- [7] Y. P. Loh and C. S. Chan, “Getting to Know Low-Light Images with the Exclusively Dark Dataset,” *Computer Vision and Image Understanding*, 2019.
- [8] C. S. Chan et al., “Exclusively Dark Image Dataset,” official repository, accessed Aug. 2026. <https://github.com/cs-chan/Exclusively-Dark-Image-Dataset>
- [9] F. Yu et al., “BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning,” in *CVPR*, 2020.
- [10] M. Cordts et al., “The Cityscapes Dataset for Semantic Urban Scene Understanding,” in *CVPR*, 2016.
- [11] I. M. Sobol, “Global Sensitivity Indices for Nonlinear Mathematical Models and Their Monte Carlo Estimates,” *Mathematics and Computers in Simulation*, vol. 55, no. 1–3, pp. 271–280, 2001.
- [12] A. Saltelli, P. Annoni, I. Azzini, F. Campolongo, M. Ratto, and S. Tarantola, “Variance Based Sensitivity Analysis of Model Output: Design and Estimator for the Total Sensitivity Index,” *Computer Physics Communications*, vol. 181, no. 2, pp. 259–270, 2010.

- [13] SALib, “Sensitivity Analysis Library in Python,” official documentation, accessed Aug. 2026. <https://salib.readthedocs.io/>
- [14] F. Wilcoxon, “Individual Comparisons by Ranking Methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [15] M. Friedman, “The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance,” *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.
- [16] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [17] J. R. McClean et al., “Barren Plateaus in Quantum Neural Network Training Landscapes,” *Nature Communications*, vol. 9, article 4812, 2018.